



BSc (Hons) in **Information Technology**
Specialized in AI
IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{ 'wall_ahead': True/False, 'food_here': True/False }` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A mathematically perfect Table-Driven Agent is impractical because it would need to store an action for every possible percept sequence. In complex environments such as Chess, the number of possible states and percept sequences becomes extremely large. As the agent's lifetime increases, the number of possible histories grows rapidly, causing combinatorial explosion. Therefore, storing and searching the complete table would require an unrealistic amount of memory and computation.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The Simple Reflex Agent implements Condition-Action rules through conditional statements in the `sense_and_act()` method. Each rule checks the current percept and immediately selects an appropriate action. For example, the condition `percept["smells_toxin"]` checks whether a toxic trap is detected, and if it is true, the agent selects the Down action. Similarly, the condition `percept["food_here"]` causes the agent to move Up when food is detected. When a wall is detected in the upward direction, the agent selects an alternative direction such as Right, Left, or Down. If none of these conditions are satisfied, the agent continues moving Up. These rules demonstrate the Condition-Action structure because the agent directly maps its

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The Simple Reflex Agent can become trapped in an infinite loop because it makes decisions only from its current percept and does not maintain any percept history or internal state. The environment used in this practical is partially observable, meaning that the agent cannot access complete information about the environment or remember locations that it has previously visited. When the agent reaches a corner, obstacle, or repeated configuration, it may receive the same percept again. Since it has no memory to recognize that the situation has already occurred, it applies the same Condition-Action rule repeatedly. Therefore, the combination of partial observability and the absence of percept history can cause the agent to repeat actions, become trapped in a cycle, or fail to find an alternative path.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The Model-Based Agent improves upon the Simple Reflex Agent by maintaining an internal state that represents information about the environment that is not directly available from the current percept. In the implementation, the agent maintains its estimated position using the position variable and records previously visited locations using the `visited_cells` set. The Sensor Model is represented by the percept information received from the environment, including whether walls, food, or toxic traps are present. The Transition Model is represented by the `next_position()` and `update_position()` methods, which determine how the agent's internal position changes as a result of an action and verify whether the movement is blocked. By combining the current percept with its stored internal state, the Model-Based Agent can identify previously visited cells and select alternative actions. This allows the agent

Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING